

Lane Detection Pipeline (Lightweight OpenCV)

Context

This pipeline is designed for:

- ROS 2 (Humble)
 - CARLA 0.9.16
 - Low-resource system (CPU only, ~10 FPS target)
 - Input: RGB image (160x120)
-

1. Grayscale Conversion

Convert RGB image to grayscale:

$$I(x, y) = 0.299R + 0.587G + 0.114B$$

Purpose

- Reduce computation (3 channels $\rightarrow$ 1)
 - Focus on intensity rather than color
-

2. Gaussian Blur

Apply Gaussian filter:

$$I_{blur} = I * G_{\sigma}$$

Where:

- G_{σ} : Gaussian kernel
- Typical kernel: (5x5)

Purpose

- Remove noise
 - Improve edge detection stability
-

3. Canny Edge Detection

Compute gradients:

$$G = \sqrt{G_x^2 + G_y^2}$$

Apply thresholding:

- Strong edges kept
- Weak edges suppressed

Purpose

- Extract edges from image
 - Reduce data complexity
-

4. Region of Interest (ROI)

Define a polygon mask (trapezoid):

$$\text{ROI} = \text{mask}(I_{\text{edges}})$$

Purpose

- Focus on road area only
 - Remove irrelevant regions (sky, buildings)
-

5. Hough Line Transform

Convert edge points into line representation:

$$\rho = x \cos \theta + y \sin \theta$$

Each edge point votes for possible lines in parameter space.

Purpose

- Detect straight line segments
 - Convert pixel clusters into structured lines
-

6. Lane Separation (Left / Right)

Slope of a line:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

Classification:

- Left lane $\rightarrow m < 0$
- Right lane $\rightarrow m > 0$

Reject near-horizontal lines:

$$|m| < 0.5 \rightarrow \text{discard}$$

Purpose

- Separate lanes
 - Remove noise lines
-

7. Line Averaging (Stabilization)

Fit a line using least squares:

$$y = mx + b$$

Using points from detected segments.

Solve using:

$$(m, b) = \text{polyfit}(x, y)$$

Purpose

- Smooth noisy detections
 - Produce stable lane lines
-

8. Lane Center Calculation

Compute lane center:

$$x_{center} = \frac{x_{left} + x_{right}}{2}$$

Vehicle center:

$$x_{vehicle} = \frac{width}{2}$$

Offset:

$$offset = x_{center} - x_{vehicle}$$

Interpretation

- Offset < 0 → vehicle is left of lane center
- Offset > 0 → vehicle is right of lane center

9. Optional Smoothing (Temporal Filter)

Exponential moving average:

$$offset_{smooth} = \alpha \cdot offset_{prev} + (1 - \alpha) \cdot offset$$

Where:

- $\alpha \in [0, 1]$, e.g. 0.8

Purpose

- Reduce jitter
- Improve control stability

Pipeline Summary

```
RGB Image
↓
Grayscale
↓
Gaussian Blur
↓
Canny Edge Detection
↓
ROI Masking
↓
Hough Transform
↓
```

Lane Separation

↓

Line Fitting

↓

Lane Center

↓

Offset (for control)

Notes for Low-Resource Systems

- Use low resolution (160x120)
- Avoid deep learning models
- Keep kernel sizes small
- Limit Hough parameters
- Avoid GUI operations (cv2.imshow)

Next Step

Use computed offset as input to:

- PID Controller → Steering command